



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE — FALL 2021

HOMEWORK 4 - SOLUTION

Due: 11:59pm, Saturday, April 17

Submit to: engin604assignments@gmail.com

Maximum page limit: 5 pages

See the homework policies at <https://docenciaweb.fen.uchile.cl>



The file `ETFs.xlsx` contains the OHCL (*Open, High, Close and Low*) of the following *Exchange Traded Funds*¹ at a daily frequency:

- SPY** (S&P500 ETF), from 2012-11-26 to 2020-12-30
- EFA** (a non-US equities ETF), from 2012-12-31 to 2020-12-30
- IJS** (a small-cap value ETF), from 2012-10-31 to 2020-12-30
- EEM** (an emerging-markets ETF), from 2012-04-25 to 2020-12-30
- AGG** (A bond ETF), from 2012-12-31 to 2020-12-30

All the data were extracted from *Yahoo Finance* using the `YFinance` library.

- Load the file `ETFs.xlsx` into your workspace so that each sheet is stored in a dictionary.

```
import pandas as pd # load pandas
import numpy as np # load numpy
from datetime import datetime # load datetime

# read etfs into a dictionary
etfs = pd.read_excel('ETFs.xlsx', sheet_name=None, index_col=0)
```

- Build a `DataFrame` containing the closing price of each ETF.

- The column (`columns`) must have the name of the ETF and the index (`index`) the date.
- Work only with the data from 2012-12-31 to 2020-12-30.

Explain at least two ways of doing (2.b).

```
# empty list
etfs_list = []

# store in a list the dfs with the closing price
for etf_name, etf_price in etfs.items():
    etfs_list.append(etf_price.loc[:, ['Close']].rename(columns={'Close': etf_name}))

# concatenate the list
etfs_daily = pd.concat(etfs_list, axis=1)
```

¹These are collective investment funds whose investment policy consists of replicating an index.

3. Transform the DataFrame created in (2) to a monthly frequency. This is done by assuming that the last available price of the month corresponds to the monthly price.

```
# extract the year from the date in the index
etfs_daily['Year'] = etfs_daily.index.year

# extract the month from the date in the index
etfs_daily['Month'] = etfs_daily.index.month

# extract the day from the date in the index
etfs_daily['Day'] = etfs_daily.index.day

# remove the time from the date in the index
etfs_daily.index = etfs_daily.index.date

# drop the na so that the df starts from 2012-12-31
etfs_daily.dropna(inplace=True)

# group and take the last price as the monthly one
etfs_monthly = etfs_daily.groupby(['Year', 'Month']).tail(1)
```

4. Using a lambda (anonymous) function, generate the monthly logarithmic return for each ETF. Do not forget to drop the row of NAs.

$$R_{j,t} = \ln \left(\frac{P_{j,t}}{P_{j,t-1}} \right) \quad j = 1, \dots, 5 \quad t = 1, \dots, 97$$

Hint: NumPy has the `np.log()` function to compute the natural logarithm.

```
# define the function
def logret(x):
    return(np.log(x/x.shift(1)))

# compute the logarithmic return
etfs_monthly_log_return = etfs_monthly.loc[:, 'SPY': 'AGG'].apply(lambda x: logret(x))

# drop the row with na
etfs_monthly_log_return.dropna(inplace=True)
```

5. Write a function that computes a brief descriptive statistic containing: arithmetic mean, standard deviation, *kurtosis*, *skewness* and Sharpe ratio (assume a risk-free rate equal to 0). For the function, take the following into account:

- The *input* must be a Pandas with a Series structure.
- The *output* must be a Pandas with a Series structure where:
 - The *name* attribute contains the name of the *input*.
 - The *index* attribute contains the name of each descriptive statistic computed.

Apply the function to the DataFrame created in (4) by means of a lambda (anonymous) function.

```
# define the function for the descriptive statistics
def desc_stat(x):

    mean = sum(x)/len(x)
    std = (sum((x-mean) ** 2)/(len(x)-1)) ** (1/2)
    kurtosis = (sum((x-mean) ** 4)/len(x))/((sum((x-mean) ** 2)/len(x)) ** 2)
    skewness = (sum((x-mean) ** 3)/len(x))/((sum((x-mean) ** 2)/len(x)) ** (1.5))
```

```

output = pd.Series(data=[mean, std, kurtosis, skewness],
                    index=['mean', 'std', 'kurtosis', 'skewness'])

return(output)

# apply the function
summary = etfs_monthly_log_return.apply(lambda x: desc_stat(x))

```

6. Using a line chart, plot the cumulative logarithmic return of each ETF.

```

# plot the cumulative return of each etf
etfs_monthly_log_return.loc[:, 'SPY': 'AGG'].apply(lambda x: x.cumsum()).plot()

```

7. The tangency portfolio is the one that maximizes the Sharpe ratio. The optimization problem expressed in its matrix form is:

$$\max_{\mathbf{t}} \frac{\mathbf{t}'\boldsymbol{\mu} - r_f}{(\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t})^{\frac{1}{2}}} = \frac{\mu_{p,t} - r_f}{\sigma_{p,t}} \quad s.t. \quad \mathbf{t}'\mathbf{1} = 1$$

Whose solution (also in matrix form) is:

$$\mathbf{t} = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \cdot \mathbf{1})}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r_f \cdot \mathbf{1})}$$

Where $\mathbf{t}$ is the vector of optimal weights of the tangency portfolio, $\mathbf{1}$ a vector of ones whose dimension is $j \times 1$, with j the total number of assets, and $\boldsymbol{\Sigma}^{-1}$ the inverse of the variance-covariance matrix ($j \times j$).

Using the matrix solution for $\mathbf{t}$, write a function whose *inputs* are: the vector of average returns, the variance-covariance matrix of the assets (ETFs) that would make up the tangency portfolio and the risk-free rate (scalar).

The *output* of the function must be the weights $\mathbf{t}$, the expected return and the standard deviation of the tangency portfolio.

- The expected return of the tangency portfolio is:

$$R_{p,t} = \mathbf{t}' \cdot \boldsymbol{\mu}$$

Where $\boldsymbol{\mu}$ is the vector of average returns of dimension $j \times 1$.

- The standard deviation of the minimum-variance portfolio is:

$$\sigma_{p,t} = \sqrt{\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t}}$$

Test the function using the average returns created in (5) and build the variance-covariance matrix using the DataFrame generated in (4). Explain your function line by line. Assume a risk-free rate equal to 0.005.

Hint: To convert rows or columns of a DataFrame to NumPy, use the `.to_numpy()` attribute after the selection (`loc` or `iloc`).

```

# extract the mean row created with the function and convert it to a numpy array
Er_mat = summary.loc[['mean'],:].to_numpy()

# compute the variance-covariance matrix
Cov_mat = np.cov(etfs_monthly_log_return, rowvar=False)

# define the function that computes the tangency portfolio
def tangency_port(mu, cov, rf):

```

```

m,n = mu.shape

one_vec = np.ones((n,1))
sigma_inv_mat = np.linalg.inv(cov)

mu_minus_rf = mu - rf

top_mat = sigma_inv_mat.dot(mu_minus_rf.T)
bottom_escalar = one_vec.T.dot(top_mat)

wi = top_mat / bottom_escalar
er = mu.dot(wi)
sigma2 = (wi.T.dot(cov).dot(wi)) ** (1/2)

return([wi, er, sigma2])

# risk-free rate
rf = 0.005

# compute the tangency portfolio
tangency_port(Er_mat, Cov_mat, rf)

```